FIAP ON

IA COMO FERTILIZANTE DIGITAL - UM NOVO AGRONEGÓCIO DO AMANHÃ

CÓDIGO ORGANIZADO, PROJETO SEGURO: POO NA PRÁTICA COM ARDUINO E ESP32

8

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Circuito do exemplo do pisca usando passagem de parâmetros 18
Figura 2 - Circuito do exemplo de código com classe 26

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Primeiro módulo em Python sendo criado 5
Código-fonte 2 - Importando o módulo no código principal, main 6
Código-fonte 3 - Pisca-pisca tradicional 7
Código-fonte 4 - Exemplo pisca-pisca usando POO 8
Código-fonte 5 - Sintaxe de uma função 11
Código-fonte 6 - Exemplo de função 12
Código-fonte 7 - Chamando uma função dentro do void setup() 13
Código-fonte 8 - Exemplo de função do tipo void, isto é, sem return 14
Código-fonte 9 - Função sem parâmetro 14
Código-fonte 10 - Função com parâmetro 14
Código-fonte 11 - Função com retorno e sem parâmetro 15
Código-fonte 12 - Função com retorno e com parâmetro 15
Código-fonte 13 - Exemplo de passagem de valor 16
Código-fonte 14 - Exemplo de passagem de parâmetro 16
Código-fonte 15 - Exemplo do pisca usando passagem de parâmetros 17
Código-fonte 16 - Sintaxe de uma classe em C/C++ 20
Código-fonte 17 - Exemplo de private 21
Código-fonte 18 - Exemplo de public 21
Código-fonte 19 - Exemplo de construtor 22
Código-fonte 20 - Exemplo de métodos 22
Código-fonte 21 - Exemplo completo de classe 23
Código-fonte 22 - Instanciando o objeto no código principal void loop() 23
Código-fonte 23 - Exemplo de código sem classe 24
Código-fonte 24 - Exemplo de código com classe 25
Código-fonte 25 - Reforçando o entendimento do método 27
Código-fonte 26 - Encapsulamento de métodos 28
Código-fonte 27 - Exemplo prático de classe usando sensor de temperatura 29
Código-fonte 28 - Instanciando o objeto no código principal 30
Código-fonte 29 - Declarando um construtor 31
Código-fonte 30 - Outro exemplo de Construtor na prática 32
Código-fonte 31 - Instanciando o objeto motorA no código principal 33
Código-fonte 32 - Exemplo de destrutor 34
Código-fonte 33 - Exemplo de depuração usando Monitor Serial da IDE 35

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 CÓDIGO ORGANIZADO, PROJETO SEGURO: POO NA PRÁTICA COM ARDUINO E ESP32 5
1.1 Introdução 5
1.1.1 Paralelo entre Programação Modular em Python e C/C++ 5
1.1.2 Diferença entre Programação Estruturada e Programação Orientada a Objeto 6
1.1.3 Vantagens e Desvantagens do POO na Arquitetura do ESP32 8
2 FUNÇÕES 11
2.1 O que são Funções? 11
2.2 Estrutura Básica de uma Função em C/C++ 11
2.3 Tipos de Funções no Arduino 14
2.4 Passagem de parâmetros 15
3 CLASSES E OBJETOS 19
3.1 Estrutura de uma classe 20
3.2 Criando uma classe no Arduino 22
3.3 Instanciando objetos no código principal 23
3.4 Comparação: Código com e sem classes 24
4 MÉTODOS E ENCAPSULAMENTOS 27
4.1 O que são métodos dentro de uma classe? 27
4.2 Encapsulamento e boas práticas 28
4.3 Exemplo prático: Criando uma classe Sensor para leitura de temperatura 29
5 CONSTRUTORES E DESTRUTORES 31
5.1 O que é um Construtor? 31
5.2 Como declarar e utilizar um construtor 31
5.3 O que é um destrutor e quando usá-lo? 33
5.4 Testes e depuração no Arduino IDE ou Wokwi 34
CONCLUSÃO 37
REFERÊNCIAS 38

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 CÓDIGO ORGANIZADO, PROJETO SEGURO: POO NA PRÁTICA COM ARDUINO E ESP32

1.1 Introdução

Para entender sobre Funções, Classes e Métodos em linguagem C/C++, primeiramente, precisamos entender o que é **programação modular**.

A programação modular é uma abordagem que divide um programa em partes menores e independentes, chamadas de módulos. Cada módulo é responsável por uma tarefa específica, o que facilita a leitura, manutenção e reutilização do código. No contexto do Arduino/ESP, isso significa criar funções e classes para organizar o código, evitando repetições e tornando-o mais eficiente.

Outras linguagens famosas, como o Python, por exemplo, já fazem isso. Em Python, a modularização é feita principalmente por meio de:

- **Funções:** blocos de código reutilizáveis que executam tarefas específicas.
- **Módulos:** arquivos ".py" que podem ser importados em outros códigos.
- **Pacotes:** pastas que agrupam vários módulos, tornando o projeto mais organizado.
- **Classes:** para aplicar o conceito de Programação Orientada a Objetos (POO).

1.1.1 Paralelo entre Programação Modular em Python e C/C++

O primeiro passo para programar de forma modular em Python é criar uma função em um módulo, que é um arquivo Python a parte, chamado por exemplo, **util.py**.

```
def saudacao(nome):
    return f"Olá, {nome}!"
```

Código-fonte 1 - Primeiro módulo em Python sendo criado
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Já no segundo passo, será necessário importar e utilizar o módulo no arquivo principal **main.py**. Os dois arquivos (util.py e main.py) precisam estar na mesma pasta do projeto, caso não estejam, a pasta utilizada para manter o util.py precisa estar mapeada no seu path do sistema para que o **import** funcione no main.py.

```
import util

print(util.saudacao("Maria"))
```

Código-fonte 2 - Importando o módulo no código principal, main
Fonte: Elaborado pelo autor (2025)

A saída, será **Olá, Maria!** Note que houve uma transferência de parâmetro “Maria” para dentro da função, que por sua vez, a herdou, juntou com “Olá” e retornou a resposta pronta. Em C/C++ acontecerá a mesma coisa.

Portanto, tanto em Python quanto em C/C++, a modularização segue os mesmos princípios e é importante porque permite:

- **Organização e colaboração:** facilita a leitura e manutenção do código.
- **Reutilização:** módulos podem ser reutilizados em diferentes projetos.
- **Facilidade de testes e depuração:** cada módulo pode ser testado separadamente.
- **Escalabilidade:** projetos grandes se tornam mais fáceis de gerenciar.

1.1.2 Diferença entre Programação Estruturada e Programação Orientada a Objeto

A **programação estruturada** é baseada na criação de funções e no uso de comandos sequenciais para controlar o fluxo de execução do programa. Embora eficiente para projetos menores, essa abordagem pode se tornar complexa e difícil de manter à medida que o código cresce. Por exemplo, um código simples que pisca um LED repetidamente pode ser escrito da seguinte forma:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void setup() {
    pinMode(13, OUTPUT);
}

void loop() {
    digitalWrite(13, HIGH);
    delay(1000);
    digitalWrite(13, LOW);
    delay(1000);
}
```

Código-fonte 3 - Pisca-pisca tradicional
Fonte: Elaborado pelo autor (2025)

No entanto, esse código é repetitivo e pode se tornar difícil de modificar, especialmente se precisar controlar múltiplos LEDs ou alterar o comportamento do programa.

Uma boa alternativa à **Programação Estruturada** é a **Programação Orientada a Objetos (POO)**, que organiza o código em torno de classes e objetos, agrupando dados e comportamentos em estruturas organizadas.

Essa abordagem facilita a reutilização e manutenção do código, permitindo a criação de abstrações mais próximas do mundo real. Por exemplo, podemos criar uma classe para representar um LED.

Observe o código a seguir:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class Led {
  private:
    int pino;
  public:
    Led(int p) {
        pino = p;
        pinMode(pino, OUTPUT);
    }
    void piscar(int tempo) {
        digitalWrite(pino, HIGH);
        delay(tempo);
        digitalWrite(pino, LOW);
        delay(tempo);
    }
};

Led meuLed(13);

void loop() {
    meuLed.piscar(1000);
}
```

Código-fonte 4 - Exemplo pisca-pisca usando POO
Fonte: Elaborado pelo autor (2025)

Ao comparar os dois códigos (Código-fonte “Pisca-pisca tradicional” e o Código-fonte “Exemplo pisca-pisca usando POO”), em um primeiro momento, parece que o segundo é mais difícil e rebuscado de fazer e de compreender em comparação ao primeiro.

No entanto, para projetos escaláveis e mais complexos, a opção do POO é sempre melhor e mais profissional, pois você cria pequenas ferramentas secundárias em forma de classes e métodos que serão úteis ao longo do seu código principal.

Note que no pisca-pisca usando POO o void loop() está mais simplificado. Ele possui apenas uma única linha “meuLed.piscar(1000);”

É sobre isso que vamos discutir neste capítulo, isto é, como criar funções, classes e métodos de modo a profissionalizar e otimizar o seu código-fonte em C/C++.

1.1.3 Vantagens e Desvantagens do POO na Arquitetura do ESP32

A modularização no ESP32 apresenta vantagens significativas do ponto de vista de memória e recursos internos. Um dos principais benefícios é a eficiência no

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

uso da memória, já que a divisão do código em módulos reduz a duplicação de instruções e promove a reutilização de funções e classes.

Isso contribui para um consumo mais otimizado dos recursos, preservando a capacidade limitada da SRAM do ESP32. Além disso, a modularização facilita o gerenciamento da memória dinâmica, uma vez que permite um controle mais rigoroso sobre a alocação e liberação de recursos, minimizando o risco de vazamentos de memória.

Outro aspecto relevante é a melhoria na legibilidade e manutenção do código, o que torna o processo de identificação e correção de erros mais rápido e eficiente. Com módulos bem definidos, é possível isolar falhas de forma mais precisa, agilizando o processo de depuração.

Outra vantagem é que a modularização favorece o desenvolvimento escalável, permitindo a adição de novos recursos e funcionalidades sem impactar significativamente o desempenho geral do sistema.

Com a criação de módulos independentes, torna-se mais fácil incorporar novos dispositivos ou funcionalidades ao projeto, garantindo que o sistema continue robusto e organizado. Isso é particularmente relevante em aplicações complexas de IoT, onde a expansão contínua é necessária para acompanhar as demandas tecnológicas.

Por outro lado, a modularização no ESP32 também possui algumas desvantagens relacionadas ao consumo de memória. A criação excessiva de módulos e instâncias de classes pode ocupar uma quantidade significativa de SRAM, prejudicando o desempenho do sistema em projetos com restrições de recursos. Cada instância de uma classe consome memória, e o uso indiscriminado de bibliotecas adicionais pode aumentar consideravelmente o tamanho do programa, exigindo uma gestão criteriosa de recursos para evitar sobrecarga.

Além disso, a modularização exagerada pode introduzir complexidade desnecessária no fluxo de execução do código. Chamadas excessivas entre módulos podem gerar sobrecarga no processamento, afetando o tempo de resposta em aplicações que demandam alta performance e baixa latência.

Em sistemas embarcados, essa sobrecarga pode comprometer a eficiência e levar a atrasos na execução de tarefas críticas, especialmente quando múltiplos sensores e atuadores estão em operação simultânea.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Portanto, ao modularizar um projeto no ESP32, é fundamental equilibrar a organização do código com a eficiência no uso dos recursos. Isso significa criar módulos claros e bem definidos, e evitar a criação desnecessária de classes ou a importação excessiva de bibliotecas.

O uso adequado de funções e a otimização das classes são estratégias importantes para garantir que o código seja eficiente tanto em termos de memória quanto de processamento.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 FUNÇÕES

2.1 O que são Funções?

Em C/C++, **uma função** é um bloco de código reutilizável que realiza uma tarefa específica que você vai criar no seu projeto.

No contexto do Arduino, as funções permitem organizar o código, tornando-o mais limpo, legível e de fácil manutenção. Elas ajudam a evitar repetições e facilitam o processo de depuração.

Quando o programa é executado, uma função só é ativada quando chamada pelo nome dentro do **void loop()**, **void setup()** ou dentro de uma função anterior que você criou, mas que foi primeiramente carregada no void setup() ou void loop(), o que proporciona maior controle sobre o fluxo de execução.

As funções são especialmente úteis em projetos de Arduino, pois permitem isolar funcionalidades, como acionar um LED, ler um sensor ou controlar um motor. Ao dividir o código em funções, é possível focar no desenvolvimento de cada parte de forma independente, promovendo uma melhor organização e reduzindo o risco de erros.

2.2 Estrutura Básica de uma Função em C/C++

Uma função em C/C++ possui uma estrutura básica que inclui o tipo de retorno, o nome da função, os parâmetros e o bloco de código que define o comportamento da função. A sintaxe é a seguinte:

```
tipo_de_retorno nomeDaFuncao(tipo_parametro parametro) {
    // Bloco de código
    return valor; // opcional, dependendo do tipo de retorno
}
```

Código-fonte 5 - Sintaxe de uma função
Fonte: Elaborado pelo autor (2025)

Detalhando a explicação do código:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Tipo de retorno:** define a tipagem do dado que a função irá retornar. Se a função não retorna nenhum valor, utiliza-se o tipo **void**. No lugar do void, você pode usar **int**, **unsigned int**, **float**, **double**, **char**, **String**, **array**, **struct**, **int*** e **auto**.
- **Nome da função:** deve ser descritivo e seguir as regras de nomenclatura do C++, isto é, o nome da função deve começar com uma letra (ou sublinhado, embora não seja recomendado para nomes de funções comuns) e pode conter letras, números e sublinhados. Porém, não pode conter espaços ou caracteres especiais. É recomendável usar a convenção **camelCase** para funções, começando com letra minúscula (exemplo: **calcularMedia**), enquanto nomes de classes geralmente seguem o padrão **PascalCase** (exemplo: **SensorTemperatura**). Os nomes devem refletir claramente a ação ou propósito da função, sendo curtos, mas informativos. Além disso, devem evitar o uso de palavras reservadas da linguagem e abreviações excessivas que prejudiquem o entendimento.
- **Parâmetros:** são variáveis que recebem valores durante a chamada da função. Podem ser do tipo int, float, char, entre outros. Se a função não necessita de parâmetros, os parênteses permanecem vazios.
- **Bloco de código:** contém as instruções criadas por você que serão executadas quando a função for chamada.
- **Return:** utilizado para retornar um valor ao final da execução da função, quando necessário.

Exemplo de uma função simples que retorna a soma de dois números:

```
int somar(int a, int b) {
    return a + b;
}
```

Código-fonte 6 - Exemplo de função
Fonte: Elaborado pelo autor (2025)

Vamos analisar a construção da função em C++:

- **int** (tipo de retorno): este é o tipo de dado que a função irá retornar. No caso, a função “somar” está definida para retornar um valor do tipo “int” (inteiro).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Isso significa que, após a execução, ela entregará um número inteiro como resultado.

- **somar** (nome da função): este é o identificador da função. O nome deve ser descritivo e indicar claramente o que a função faz. Nesse caso, o nome "somar" sugere que a função realiza uma operação de soma.
- **(int a, int b)** (parâmetros da função): entre parênteses e negrito, são definidos os **parâmetros de entrada** que a função precisa para executar sua tarefa. Neste exemplo, "a" e "b" são dois parâmetros do tipo "int", e isso significa que a função espera receber dois números inteiros quando for chamada. Os nomes "a" e "b" são variáveis locais e podem ser qualquer identificador válido.
- **{ ... }** (corpo da função): o corpo da função está delimitado pelas chaves "{ }". É dentro desse bloco que o comportamento da função é definido.
- **return a + b;** (instrução de retorno): a palavra-chave "return" indica o valor que a função deve retornar após a execução. Neste caso, ela retorna o resultado da soma de "a" e "b". Como o tipo de retorno da função é "int", o valor retornado também deve ser um inteiro.

Para utilizar essa função, basta chamá-la e fornecer os dois valores inteiros como argumentos. Será necessário declarar localmente a variável que vai armazenar o resultado que virá da função "somar", que nesse caso é "resultado" e sua tipagem é "int", para casar com o tipo da função "somar".

```
void setup(){
int resultado = somar(5, 3);
}
```

Código-fonte 7 - Chamando uma função dentro do void setup()
Fonte: Elaborado pelo autor (2025)

Neste exemplo, a função "somar" recebe os valores "5" e "3", processa a soma e retorna o resultado "8", que é armazenado na variável "resultado".

Vejamos um outro exemplo de função, mas agora do tipo void para piscar um LED:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
void piscarLed(int pino, int tempo) {
    digitalWrite(pino, HIGH);
    delay(tempo);
    digitalWrite(pino, LOW);
    delay(tempo);
}

void setup(){
     pinMode(13, OUTPUT);
}

void                                        loop(){
piscarLed(13,1000);
}
```

Código-fonte 8 - Exemplo de função do tipo void, isto é, sem return
Fonte: Elaborado pelo autor (2025)

Neste segundo exemplo, a função não retorna nenhum valor, apenas executa uma ação. Por isso, uma função do tipo **void não precisa de um return**.

2.3 Tipos de Funções no Arduino

No Arduino/ESP, as funções podem ser classificadas de acordo com o tipo de retorno e a necessidade de parâmetros.

- **Funções void sem parâmetros:** estas funções não retornam valor e não recebem parâmetros. São utilizadas para executar ações fixas.

```
void acenderLed() {
    digitalWrite(13, HIGH);
}
```

Código-fonte 9 - Função sem parâmetro
Fonte: Elaborado pelo autor (2025)

- **Funções void com parâmetros:** permitem passar informações para a função durante sua chamada, tornando o código mais flexível.

```
void acenderLed(int pino) {
    digitalWrite(pino, HIGH);
}
```

Código-fonte 10 - Função com parâmetro
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Funções com retorno e sem parâmetros:** retornam um valor após a execução, mesmo sem receber dados de entrada.

```
int lerSensor() {
    return analogRead(A0);
}
```

Código-fonte 11 - Função com retorno e sem parâmetro
Fonte: Elaborado pelo autor (2025)

- **Funções com retorno e com parâmetros:** recebem dados de entrada e retornam um valor com base no processamento realizado.

```
float calcularMedia(float valor1, float valor2) {
    return (valor1 + valor2) / 2;
}
```

Código-fonte 12 - Função com retorno e com parâmetro
Fonte: Elaborado pelo autor (2025)

A utilização adequada de cada tipo de função contribui para a criação de códigos mais organizados, eficientes e fáceis de expandir.

2.4 Passagem de parâmetros

Em C/C++, os parâmetros de uma função podem ser passados de duas maneiras: por valor e por referência.

Na **passagem por valor**, uma cópia do valor do argumento é passada para a função. Isso significa que qualquer alteração feita dentro da função não afeta o valor original da variável. Esse método é seguro, pois evita modificações acidentais nos dados originais, mas pode ser menos eficiente se o argumento for uma estrutura grande, já que a cópia consome mais memória e processamento.

Vejamos um exemplo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     void alterarValor(int x) {
         x = 20; // Altera apenas a cópia local
     }

     void setup(){
     int numero = 10;
     alterarValor(numero); // a var "numero" ainda será 10 porque
alterarValor não mexeu em "numero"
}
```

Código-fonte 13 - Exemplo de passagem de valor.
Fonte: Elaborado pelo autor (2025)

O que aconteceu no exemplo anterior? Ele declarou a variável “numero” localmente com o valor 10, mas carregou a função “alterarValor” com o conteúdo de “número”, e, na função “alterarValor”, o x está com 20. Mas “x” é uma variável local dentro da função alterarValor, que não vai mudar o conteúdo de “número” que está local na função setup(). Ou seja, esse exemplo é bastante básico, porém, é bom entender essas questões para dominar a transferência de argumentos.

Já na **passagem por referência**, a função recebe o endereço da variável original. Isso significa que qualquer modificação realizada dentro da função afeta diretamente o valor da variável fora da função. Esse método é eficiente em termos de memória, pois evita a cópia de grandes estruturas de dados, mas requer maior cuidado para não modificar valores inadvertidamente.

A diferença bem sutil é o uso do “&” (E comercial) como argumento da função “alterarValor”. O “&” indica que estamos pegando a posição de memória da variável “número”, e assim, podemos modificar o conteúdo da variável “número” de 10 para 20 após chamar a função “alterarValor”.

```
     void alterarValor(int &x) {
         x = 20; // Modifica a variável original
     }

     void setup(){
     int numero = 10;
     alterarValor(numero);
     // numero agora será 20
     }
```

Código-fonte 14 - Exemplo de passagem de parâmetro
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Entender essas diferenças é fundamental para otimizar o uso de memória e garantir a integridade dos dados em projetos com recursos limitados, como no ESP32.

Agora iremos ver mais um exemplo de passagem de valores aplicado ao pisca-pisca. Neste exemplo, vamos utilizar a passagem por referência para alterar o tempo de piscada de um LED diretamente dentro da função.

```
uint8_t ledPin = 23;      // Pino do LED
uint8_t botaoPin = 22;     // Pino do botão de pressão
uint16_t tempoPisca = 500; // Tempo inicial de pisca-pisca
bool estadoBotaoAnterior = LOW;

void ajustarTempo(uint16_t &tempo) {
  tempo += 500;  //Incrementa o tempo a cada pressionamento
  if (tempo > 2000) { //fixa um limite no pisca-pisca
    tempo = 500;
  }
}

void piscarLed(int pino, int tempo) {
  digitalWrite(pino, HIGH);
  delay(tempo);
  digitalWrite(pino, LOW);
  delay(tempo);
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  pinMode(botaoPin, INPUT_PULLUP);
}

void loop() {
  bool estadoBotao = digitalRead(botaoPin);
  if (estadoBotao == LOW) {
    ajustarTempo(tempoPisca); // Chama a função
    Serial.println("Botão pressionado");
    delay(200); // Pequeno atraso para evitar bouncing
  }
  piscarLed(ledPin, tempoPisca); //LED pisca
}
```

Código-fonte 15 - Exemplo do pisca usando passagem de parâmetros
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

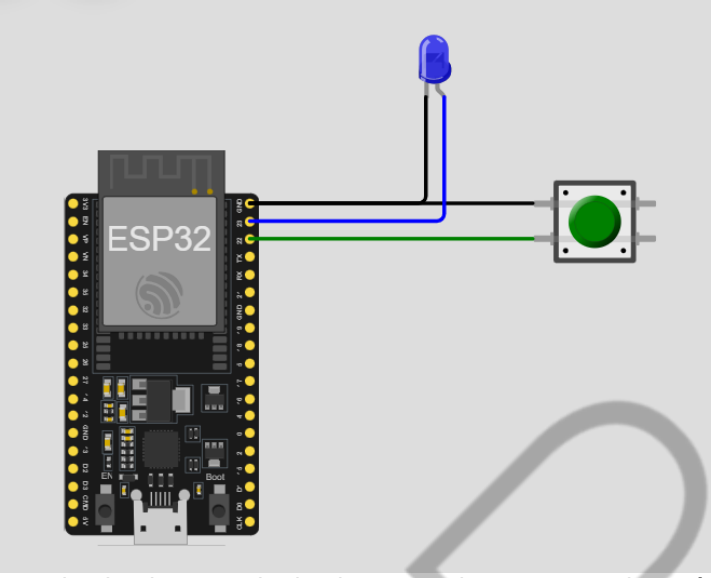

Figura 1 - Circuito do exemplo do pisca usando passagem de parâmetros
Fonte: Elaborado pelo autor (2025)

Explicando o código anterior, o botão está conectado ao pino sete e usa INPUT_PULLUP para evitar a necessidade de um resistor externo e da entrada de ruído quando o botão não estiver pressionado.

Quando o botão for pressionado, a função ajustarTempo() incrementa 100ms no tempo do pisca-pisca.

Se o tempo ultrapassar 2000ms, ele volta para 500ms, garantindo que o LED não fique muito lento.

O código detecta apenas a transição do botão de HIGH para LOW para evitar que o tempo seja alterado várias vezes rapidamente (*debouncing* simplificado).

A função piscarLed() controla o LED de acordo com o tempo atualizado, e assim, sempre que o botão for pressionado, o tempo do pisca-pisca aumenta e depois reseta ao atingir 2000ms.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 CLASSES E OBJETOS

A **Programação Orientada a Objetos** (POO) surgiu com a **evolução** da **Programação Estruturada**. Por isso, a POO é considerada um **paradigma de programação**. Um paradigma de programação representa um conjunto de conceitos, princípios e padrões que guiam a forma como um programa é escrito e executado. No caso da POO, o código é organizado em **Classes**.

Uma classe em C++ segue uma estrutura básica composta por três partes principais:

- Atributos;
- Métodos;
- Modificadores de acesso.

Os atributos armazenam os dados do objeto, os métodos definem o comportamento e os modificadores de acesso controlam a visibilidade das informações dentro e fora da classe.

Mas, e o Objeto? Onde se encaixa nessa história? Um objeto é uma instância de uma classe, ou seja, uma concretização do modelo definido pela classe. Enquanto a classe é um conceito abstrato que define atributos (dados) e métodos (comportamentos), o **objeto** é uma entidade real que ocupa espaço na memória e pode ser manipulada no programa.

Podemos fazer uma analogia utilizando um molde de bolo e bolo assado, por exemplo. A classe é como o molde: define a forma, porém, por si só, não é considerado um bolo. Já o objeto é o bolo concreto que foi feito a partir daquele molde. Cada bolo pode ter pequenas variações (diferentes ingredientes e sabores), mas todos seguem a estrutura do molde.

Portanto, a classe define os atributos (variáveis) e os métodos (funções) que o objeto terá. Um objeto, por sua vez, é considerado uma instância concreta de uma classe, ocupando memória e podendo interagir com outros elementos presentes no programa.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Por exemplo, em um projeto de automação com ESP32, podemos criar uma classe chamada Sensor para representar sensores de temperatura, luz ou umidade. Cada sensor pode ser um objeto dessa classe, com propriedades e comportamentos específicos, permitindo um código mais organizado e reutilizável.

3.1 Estrutura de uma classe

Como mencionado, uma classe em C++ segue uma estrutura básica composta por três partes principais: **atributos**, **métodos** e **modificadores de acesso**. Os atributos armazenam os dados do objeto, os métodos definem o comportamento e os modificadores de acesso controlam a visibilidade das informações dentro e fora da classe.

A sintaxe básica de uma classe em C++ é:

```
class NomeDaClasse {
  private:
    // Atributos privados
  public:
    // Construtor
    NomeDaClasse();
    // Métodos públicos
    void metodo();
};
```

Código-fonte 16 - Sintaxe de uma classe em C/C++
Fonte: Elaborado pelo autor (2025)

- **private:** o modificador “private” restringe o acesso direto aos atributos e métodos de uma classe, garantindo o princípio do encapsulamento. Isso impede que partes externas do programa alterem diretamente o estado de um objeto, evitando comportamentos inesperados ou inconsistências no código. Portanto, o uso do “private” é importante, pois:
  - Protege os atributos internos de modificações não controladas.
  - Força o uso de métodos para alterar os valores de forma segura.
  - Reduz o risco de erros causados por acessos indevidos.

  Exemplo:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class Sensor {
  private:
    int leitura; // Apenas a classe pode modificar esse valor
};
```

Código-fonte 17 - Exemplo de private
Fonte: Elaborado pelo autor (2025)

- **public:** o modificador “public” permite que os atributos e os métodos sejam acessados externamente, servindo como a interface pública da classe. Em geral, apenas métodos são declarados como públicos, enquanto atributos permanecem privados para garantir encapsulamento. Resumindo:
  - Permite o acesso controlado aos atributos através de métodos.
  - Facilita a reutilização do código, pois os métodos públicos definem como a classe será utilizada.
  - Define a interface clara para o uso da classe.

  Exemplo:

```
class Sensor {
  private:
    int leitura;
  public:
    void definirLeitura(int valor) {
        leitura = valor;
    }
};
```

Código-fonte 18 - Exemplo de public
Fonte: Elaborado pelo autor (2025)

- **Construtor:** o construtor é um método especial que é chamado automaticamente sempre que um objeto é instanciado. Ele serve para inicializar valores e garantir que o objeto esteja pronto para o uso. Resumindo:
  - Garante que os atributos tenham valores iniciais adequados.
  - Permite que o objeto seja configurado logo na sua criação.
  - Evita que o usuário do objeto esqueça de inicializar valores essenciais.

  Exemplo:

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class Sensor {
  private:
    int leitura;
  public:
    Sensor() {  // Construtor
        leitura = 0;
    }
};
```

Código-fonte 19 - Exemplo de construtor
Fonte: Elaborado pelo autor (2025)

- **Métodos:** os métodos definem o que um objeto pode fazer. Eles são funções dentro da classe e podem manipular seus atributos e interagir com outras partes do código. Em resumo:
  - Definem o comportamento do objeto.
  - Permitem manipular os atributos de forma controlada.
  - Tornam o código mais modular e reutilizável.

  Exemplo:

```
class Sensor {
  private:
    int leitura;
  public:
    Sensor() { leitura = 0; }
    void definirLeitura(int valor) {
     leitura = valor;
    }
    int obterLeitura() {
     return leitura;
    }
};
```

Código-fonte 20 - Exemplo de métodos
Fonte: Elaborado pelo autor (2025)

Note que, sempre no final de uma classe, encerra-se com um ponto e vírgula após a chave final.

3.2 Criando uma classe no Arduino

Vamos para um exemplo completo de como criar uma classe no C++ para Arduino/ESP. Para implementar a POO no Arduino, começamos definindo uma classe

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

que encapsula atributos e métodos. Vamos criar uma classe chamada **Led**, que será responsável por controlar um LED de forma modular e reutilizável.

```
class Led {
  private:
    int pino;
  public:
    Led(int p) {  // Construtor da classe
        pino = p;
        pinMode(pino, OUTPUT);
    }
    void ligar() {
        digitalWrite(pino, HIGH);
    }
    void desligar() {
        digitalWrite(pino, LOW);
    }
};
```

Código-fonte 21 - Exemplo completo de classe
Fonte: Elaborado pelo autor (2025)

O atributo **pino** é privado, garantindo que ele só possa ser modificado internamente. O construtor **Led(int p)** inicializa o LED no pino correto. O método **ligar()** ativa o LED e **desligar()** o desativa.

3.3 Instanciando objetos no código principal

Após definir a classe, podemos instanciar objetos e utilizá-los dentro do código principal.

```
Led led1(13); // Criando um objeto da classe Led no pino 13

void setup() {
    // Nenhuma configuração adicional necessária
}

void loop() {
    led1.ligar();
    delay(1000);
    led1.desligar();
    delay(1000);
}
```

Código-fonte 22 - Instanciando o objeto no código principal void loop()
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Neste cenário, o **led1** é um objeto da classe **Led**, que representa um LED conectado ao pino 13. Vale mencionar que o nome **led1** é totalmente ilustrativo, poderia ser qualquer outro nome de sua escolha. Com essa abordagem, podemos criar múltiplos LEDs sem repetir código, bastando instanciar novos objetos: Led **led2**(12); // Criando outro LED no pino 12 e agora, podemos controlar vários LEDs de maneira modular e escalável.

3.4 Comparação: Código com e sem classes

Para entender ainda mais a vantagem da POO no Arduino, vejamos a diferença entre um código sem classes (estruturado) e um código com classes (orientado a objetos). A seguir, temos um **código sem classes** (tipo, estruturado):

```
#define LED_PIN 13

void setup() {
    pinMode(LED_PIN, OUTPUT);
}

void loop() {
    digitalWrite(LED_PIN, HIGH);
    delay(1000);
    digitalWrite(LED_PIN, LOW);
    delay(1000);
}
```

Código-fonte 23 - Exemplo de código sem classe
Fonte: Elaborado pelo autor (2025)

- **Desvantagens:**
  - Se precisarmos de outro LED, teremos que duplicar o código.
  - O código se torna menos modular e reutilizável.
  - Manutenção mais difícil em projetos grandes.

Agora veremos um **código com classes** (orientado a objetos):

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class Led {
  private:
    int pino;
  public:
    Led(int p) {
        pino = p;
        pinMode(pino, OUTPUT);
    }
    void ligar() {
        digitalWrite(pino, HIGH);
    }
    void desligar() {
        digitalWrite(pino, LOW);
    }
};

Led led1(23);
Led led2(22);

void setup() {}

void loop() {
    led1.ligar();
    delay(1000);
    led1.desligar();
    delay(1000);
    led2.ligar();
    delay(500);
    led2.desligar();
    delay(500);
}
```

Código-fonte 24 - Exemplo de código com classe
Fonte: Elaborado pelo autor (2025)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

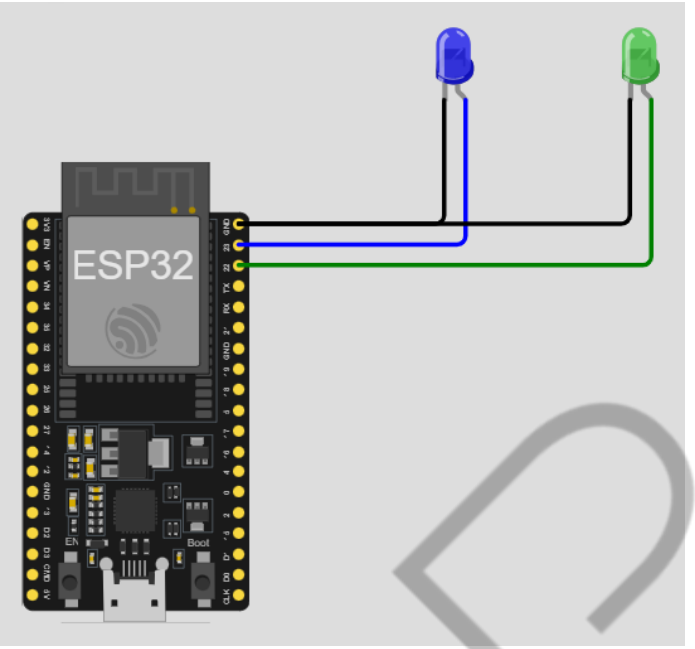

Figura 2 - Circuito do exemplo de código com classe
Fonte: Elaborado pelo autor (2025)

- **Vantagens:**
  - Código modular e reutilizável.
  - Fácil expansão para múltiplos LEDs sem duplicação de código.
  - Melhor organização e manutenção.

Portanto, como mencionado, em um primeiro momento, montar uma Classe pode parecer mais complicado, mas à longo prazo, você aprende isso com mais facilidade e rapidez, tornando o código mais limpo, flexível e eficiente, o que facilita a implementação e manutenção de projetos no Arduino e ESP32.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 MÉTODOS E ENCAPSULAMENTOS

4.1 O que são métodos dentro de uma classe?

Reforçando um pouco mais, na Programação Orientada a Objetos, métodos são funções definidas dentro de uma classe que determinam o comportamento dos objetos instanciados. Eles operam sobre os atributos da classe e permitem que um objeto realize ações específicas. Os métodos podem ser classificados de acordo com sua função:

- **Métodos públicos (*public*):** permitem interação externa, possibilitando que o usuário da classe invoque ações específicas.
- **Métodos privados (*private*):** são usados internamente na classe e não podem ser acessados diretamente de fora dela, sendo úteis para cálculos ou operações auxiliares.

A estrutura básica de um método dentro de uma classe segue o formato:

```
class Led {
  private:
    int pino;
  public:
    Led(int p) {  // Construtor da classe
        pino = p;
        pinMode(pino, OUTPUT);
    }
    void ligar() {
        digitalWrite(pino, HIGH);
    }
    void desligar() {
        digitalWrite(pino, LOW);
    }
};
```

Código-fonte 25 - Reforçando o entendimento do método
Fonte: Elaborado pelo autor (2025)

No exemplo do código-fonte anterior, **ligar()** e **desligar()** são métodos públicos que permitem controlar o LED. Eles encapsulam a lógica de acionamento, tornando o código principal mais simples e reutilizável.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4.2 Encapsulamento e boas práticas

Outro termo importante na programação C++ é o “encapsulamento” de uma classe. Embora já tenha sido visto neste capítulo, vale enaltecer sua importância. O **encapsulamento** é um dos princípios fundamentais da POO e refere-se à prática de restringir o acesso direto aos atributos de uma classe, fornecendo métodos específicos para manipulação dos dados. Isso evita modificações indesejadas e garante maior controle sobre o comportamento dos objetos.

A principal estratégia para implementar o encapsulamento é declarar os atributos como privados (*private*) e fornecer métodos públicos (*public*) para acessar ou modificar esses valores de forma controlada.

Exemplo de encapsulamento em uma classe Sensor:

```
class Sensor {
  private:
    int valorLeitura;
  public:
    Sensor() {
        valorLeitura = 0;
    }
    void definirLeitura(int leitura) {
        if (leitura >= 0) { // Garantindo um valor válido
            valorLeitura = leitura;
        }
    }
    int obterLeitura() {
        return valorLeitura;
    }
};
```

Código-fonte 26 - Encapsulamento de métodos
Fonte: Elaborado pelo autor (2025)

No exemplo do código-fonte anterior, temos o seguinte:

- O atributo **valorLeitura** é privado, impedindo acesso direto.
- O método **definirLeitura(int leitura)** permite modificar o valor apenas se ele for válido.
- O método **obterLeitura()** retorna a leitura armazenada de forma segura.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Como **boas práticas**, sempre que possível, **torne os atributos private e use métodos public para acesso controlado**.

Evite expor atributos diretamente. Em vez de sensor.valorLeitura = 100;, utilize sensor.definirLeitura(100);.

Valide os dados antes de modificá-los. Utilize verificações dentro dos métodos set para evitar valores inválidos.

Separe a interface pública da implementação interna. Isso facilita futuras alterações sem impactar o código principal.

4.3 Exemplo prático: Criando uma classe Sensor para leitura de temperatura

Para ilustrar o conceito de encapsulamento e métodos em uma classe, vamos criar uma classe SensorTemperatura que lê valores de um sensor analógico e retorna a temperatura em graus Celsius.

```
      class SensorTemperatura {
        private:
          int pinoSensor;
          float temperatura;
        public:
          SensorTemperatura(int pino) {
              pinoSensor = pino;
              temperatura = 0.0;
          }
          void atualizarLeitura() {
              int leituraAnalogica = analogRead(pinoSensor);
              temperatura = (leituraAnalogica * 5.0 / 1023.0) *
100.0; // Conversão para temperatura
          }
          float obterTemperatura() {
              return temperatura;
          }
      };
```

Código-fonte 27 - Exemplo prático de classe usando sensor de temperatura
Fonte: Elaborado pelo autor (2025)

Nesta implementação, temos o seguinte:

- **pinoSensor** é um atributo privado que armazena o pino onde o sensor está conectado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **temperatura** é um atributo privado que mantém a última leitura do sensor.
- **SensorTemperatura(int pino)** é o construtor, que inicializa o pino do sensor.
- **atualizarLeitura()** lê o valor do sensor e converte para temperatura.
- **obterTemperatura()** retorna o último valor da temperatura medida.

Agora, podemos instanciar um objeto dessa classe no código principal e utilizá-lo:

```
SensorTemperatura sensor(A0);

void setup() {
    Serial.begin(9600);
}

void loop() {
    sensor.atualizarLeitura();
    Serial.print("Temperatura: ");
    Serial.print(sensor.obterTemperatura());
    Serial.println(" °C");
    delay(2000);
}
```

Código-fonte 28 - Instanciando o objeto no código principal
Fonte: Elaborado pelo autor (2025)

Nesse código:

- Criamos um objeto sensor conectado ao pino A0.
- No **loop()**, chamamos **atualizarLeitura()** para obter a nova temperatura.
- Imprimimos a leitura no Serial Monitor a cada 2 segundos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 CONSTRUTORES E DESTRUTORES

Outro termo importante nesse capítulo, e que foi citado de forma introdutória, é o **Construtor**. Vamos aprofundar um pouco mais sobre a sua importância.

5.1 O que é um Construtor?

Um **construtor** é um método especial de uma classe em POO, automaticamente invocado no momento em que um objeto é criado. Seu papel principal é inicializar as variáveis e preparar o objeto para ser utilizado. Ao fazer uso de um construtor, você garante que os atributos tenham valores iniciais definidos e coerentes, evitando situações em que o objeto esteja incompleto ou com valores inesperados.

A inicialização automática das variáveis facilita o gerenciamento dos objetos, pois não exige que o programador atribua valores manualmente sempre que instanciar um novo objeto. Isso garante maior segurança e previsibilidade no comportamento do código, reduzindo possíveis erros decorrentes de variáveis não inicializadas.

5.2 Como declarar e utilizar um construtor

Para declarar um construtor em C++, **utilizamos o mesmo nome da classe** e não definimos um tipo de retorno, nem mesmo void. A sintaxe básica é a seguinte:

```
class NomeClasse {
  private:
    int atributo;
  public:
    NomeClasse(int valorInicial) {  // Construtor
        atributo = valorInicial;
    }
};
```

Código-fonte 29 - Declarando um construtor
Fonte: Elaborado pelo autor (2025)

Neste exemplo, o construtor recebe um parâmetro **valorInicial** para inicializar o atributo da classe.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A seguir, demonstramos um exemplo prático utilizando um construtor na criação de uma classe Motor, que representa um motor conectado a um pino específico e inicializa o estado desligado:

```
class Motor {
  private:
    int pino;
    bool estado;
  public:
    Motor(int p) {  // Construtor que recebe o pino do motor
        pino = p;
        estado = false;
        pinMode(pino, OUTPUT);
        digitalWrite(pino, LOW);
    }

    void ligar() {
        digitalWrite(pino, HIGH);
        estado = true;
    }

    void desligar() {
        digitalWrite(pino, LOW);
        estado = false;
    }

    bool obterEstado() {
        return estado;
    }
};
```

Código-fonte 30 - Outro exemplo de Construtor na prática
Fonte: Elaborado pelo autor (2025)

Agora, podemos instanciar objetos da classe Motor no código principal e utilizá-los imediatamente:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
     Motor motorA(9);   // Criando objeto motorA conectado ao
pino 9

     void setup() {
         motorA.ligar();
     }

     void loop() {
         delay(5000);
         motorA.desligar();
         delay(5000);
         motorA.ligar();
     }
```

Código-fonte 31 - Instanciando o objeto motorA no código principal
Fonte: Elaborado pelo autor (2025)

Neste exemplo:

- O construtor inicializa automaticamente o motor em estado desligado.
- O método **ligar()** ativa o motor e altera o estado interno para verdadeiro.
- O método **desligar()** desativa o motor e altera o estado interno para falso.

5.3 O que é um destrutor e quando usá-lo?

Um **destrutor** é um método especial que é automaticamente invocado quando um objeto é destruído ou sai do escopo. O papel principal do destrutor é liberar recursos que foram alocados durante o ciclo de vida do objeto, como memória dinâmica ou conexões abertas com periféricos, evitando assim vazamentos de recursos e garantindo melhor gerenciamento de memória e recursos no sistema.

No mundo do ESP, os destrutores são utilizados principalmente quando se trabalha com alocação dinâmica de memória ou quando um objeto gerencia recursos específicos que precisam ser explicitamente liberados. A sintaxe de um destrutor é semelhante à do construtor, mas precedida por um til (~). Observe:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
      class Motor {
        private:
          int pino;
        public:
          Motor(int p) {
              pino = p;
              pinMode(pino, OUTPUT);
          }

          ~Motor() {  // Destrutor
              digitalWrite(pino, LOW);   // Garante que o motor
está desligado ao destruir o objeto
          }
      };
```

Código-fonte 32 - Exemplo de destrutor
Fonte: Elaborado pelo autor (2025)

Nesse exemplo, o destrutor garante que, ao fim do uso do objeto Motor, o pino seja desativado para evitar comportamento inesperado.

No ESP, utilizar destrutores é especialmente útil em classes que gerenciam sensores, atuadores ou comunicação serial, garantindo que todos os recursos sejam adequadamente liberados ao término do objeto.

5.4 Testes e depuração no Arduino IDE ou Wokwi

Uma das principais técnicas de depuração disponíveis no Arduino IDE é o uso do Serial Monitor. O Serial Monitor permite visualizar valores de variáveis, estados de objetos e resultados de operações em tempo real, ajudando a identificar e corrigir erros rapidamente. Exemplo de uso do Serial Monitor para testes e depuração:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class Motor {
  private:
    int pino;
    bool estado;
  public:
    Motor(int p) {
        pino = p;
        estado = false;
        pinMode(pino, OUTPUT);
        digitalWrite(pino, LOW);
        Serial.begin(9600);
        Serial.println("Motor inicializado.");
    }

    void ligar() {
        digitalWrite(pino, HIGH);
        estado = true;
        Serial.println("Motor ligado.");
    }

    void desligar() {
        digitalWrite(pino, LOW);
        estado = false;
        Serial.println("Motor desligado.");
    }

    bool obterEstado() {
        Serial.print("Estado atual do motor: ");
        Serial.println(estado ? "Ligado" : "Desligado");
        return estado;
    }
};

Motor motorA(9);

void setup() {
    motorA.ligar();
    delay(2000);
    motorA.obterEstado();
    motorA.desligar();
}

void loop() {}
```

Código-fonte 33 - Exemplo de depuração usando Monitor Serial da IDE
Fonte: Elaborado pelo autor (2025)

Ao executar esse código, você pode observar as mensagens diretamente no Serial Monitor:

- Motor inicializado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Motor ligado.
- Estado atual do motor: ligado.
- Motor desligado.

Portanto, colocar mensagens estratégicas ao longo do seu código lhe traz os checkpoints do seu projeto, garantindo assim, um excelente método que divide para conquistar, isto, é um problema enorme que deve ser transformado em pequenos problemas que permitem você encontrar rapidamente pequenas soluções do todo.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Neste capítulo, exploramos em detalhes, os conceitos sobre funções, classes, métodos, construtores e destrutores aplicados ao desenvolvimento em Arduino e ESP32.

Aprendemos como a Programação Orientada a Objetos (POO) ajuda a organizar, estruturar e manter o código, especialmente em projetos maiores e mais complexos.

Além disso, também aprendemos que, através da modularização, conseguimos garantir maior clareza e reutilização do código, além de simplificar a manutenção e depuração. A criação de classes e objetos, acompanhada de métodos específicos para acesso controlado (encapsulamento), fortalece a segurança e a eficiência dos projetos.

Em relação ao uso de construtores, vimos como eles podem permitir a inicialização automática dos objetos, garantindo estados consistentes desde o momento de sua criação, enquanto os destrutores fornecem uma forma segura de liberar recursos, evitando problemas de gerenciamento de memória.

A importância dos testes e depuração também foi destacada, mostrando como ferramentas práticas, como o Serial Monitor do Arduino IDE e simulações como Wokwi, são essenciais para identificar e solucionar rapidamente problemas durante o desenvolvimento.

Por fim, a adoção dessas práticas orientadas a objetos não só aprimora a qualidade do software desenvolvido para o ESP32, mas também prepara o desenvolvedor(a) para enfrentar desafios mais complexos, típicos das modernas aplicações embarcadas e de IoT.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

AIGNER, Gerald; HÖLZLE, Urs. **Eliminating virtual function calls in C++ programs**. In: European conference on object-oriented programming. Berlin, Heidelberg: Springer Berlin Heidelberg, 1996. p. 142-166.

BRIAND, Lionel; DEVANBU, Prem; MELO, Walcelio. **An investigation into coupling measures for C++**. In: Proceedings of the 19th international conference on Software engineering. 1997. p. 412-421.

NUTARO, James J. Building software for simulation: theory and algorithms, with applications in C++. John Wiley & Sons, 2011.

RIEHLE, Dirk; BERCZUK, Stephen P. **Properties of Member Functions in C++**. 2000.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com